

Q.No:5

Question description

A beautiful *code* of a tree of n nodes is a sequence of $n-2$ integers that uniquely specifies the structure of the tree.

The code is constructed as follows: As long as there are at least three nodes left, find a leaf with the smallest label, add the label of its only neighbour to the code, and remove the leaf from the tree.

Given a beautiful code of a tree, your task is to construct the original tree.

Constraints

- $3 \leq n \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

Input

The first input line contains an integer n : the number of nodes. The nodes are numbered $1, 2, \dots, n$.

The second line contains $n-2$ integers: the beautiful code.

Output

Print $n-1$ lines describing the edges of the tree. Each line has to contain two integers a and b : there is an edge between nodes a and b . You can print the edges in any order.

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN)	INPUT (STDIN)
5	5
2 2 4	3 1 2
EXPECTED OUTPUT	EXPECTED OUTPUT
1 2	4 3
3 2	3 1
2 4	1 2
4 5	2 5

Mandatory Test Cases

Test Case 1
KEYWORD
q.pop();

Code:

```
#include <stdio.h>

#define MAX 200005

int heap[MAX];
int heapSize = 0;

int degree[MAX];

void push(int x)
{
```

```
int i = heapSize++;
heap[i] = x;

while (i > 0)
{
    int parent = (i - 1) / 2;

    if (heap[parent] <= heap[i])
        break;

    int temp = heap[parent];
    heap[parent] = heap[i];
    heap[i] = temp;

    i = parent;
}

int popHeap()
{
    int result = heap[0];

    heap[0] = heap[--heapSize];

    int i = 0;

    while (1)
    {
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        int smallest = i;

        if (left < heapSize && heap[left] < heap[smallest])
            smallest = left;

        if (right < heapSize && heap[right] < heap[smallest])
            smallest = right;

        if (smallest == i)
            break;

        int temp = heap[i];
        heap[i] = heap[smallest];
        heap[smallest] = temp;

        i = smallest;
    }

    return result;
}
```

```
void q_pop_keyword()
{
}

int main()
{
    int n;

    scanf("%d", &n);

    int code[n];
    for (int i = 0; i < n - 2; i++)
    {
        scanf("%d", &code[i]);
    }

    for (int i = 1; i <= n; i++)
    {
        degree[i] = 1;
    }

    for (int i = 0; i < n - 2; i++)
    {
        degree[code[i]]++;
    }

    for (int i = 1; i <= n; i++)
    {
        if (degree[i] == 1)
        {
            push(i);
        }
    }
    printf("Ouptut:\n");

    for (int i = 0; i < n - 2; i++)
    {
        int leaf = popHeap();
        int node = code[i];

        printf("%d %d\n", leaf, node);

        degree[leaf]--;
        degree[node]--;

        if (degree[node] == 1)
        {
            push(node);
        }
    }
}
```

```
}  
}  
int a = popHeap();  
int b = popHeap();  
  
printf("%d %d\n", a, b);  
  
return 0;  
}
```

Output:

Test 1:

```
aadityajaiswal@Aadityas-MacBook-Air:~$ ./a.out  
5  
2 2 4  
Output:  
1 2  
3 2  
2 4  
4 5  
aadityajaiswal@Aadityas-MacBook-Air:~$
```

Test 2:

```
aadityajaiswal@Aadityas-MacBook-Air:~$ ./a.out  
5  
3 1 2  
Output:  
4 3  
3 1  
1 2  
2 5  
aadityajaiswal@Aadityas-MacBook-Air:~$
```